

Supervised Learning

Feature selection

Mahdi Shafiee Kamalabad & Maarten Cruyff

Feature selection and dimensionality reduction:

1. Filter methods

- Select features before model training
- E.g. correlation, variance threshold, univariate tests

2. Wrapper methods

- Evaluate different feature subsets using a predictive model
- E.g. forward selection, backward selection, best subset selection

3. Embedded methods

- Perform feature selection during model training
- E.g. lasso, ridge, elastic net

4. Dimensionality reduction

- Replace original features with fewer new features
- E.g. principal components analysis (PCA)

Questions for discussion

- Why do we need **feature selection**?
- How is **dimensionality reduction** different?
- What problems can happen when we use too many features?

Why Feature Selection Matters: Bias–Variance Tradeoff

Consider a model with many features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

These features may include **informative predictors** - which **can reduce bias** (good)

but also **irrelevant/noisy predictors** - which **increase variance** (overfitting)

Too many features can hurt test performance.

We want to keep the good predictors and remove the bad ones. So Feature selection is about finding the signal and removing the noise. This improves:

- prediction accuracy
- interpretability
- model stability

Feature selection and bias–variance tradeoff

- Too few features $\rightarrow$ underfitting (high bias)
- Too many features $\rightarrow$ overfitting (high variance)
- Right features $\rightarrow$ best generalization
- Feature selection is a tool to control model complexity and achieve a good bias-variance tradeoff.

MSE as a fit criterion

Feature selection methods need a way to compare models.

A common choice for regression is the **mean squared error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Smaller MSE means better fit to the observed data
- Training MSE usually decreases as more features are added
- Therefore, training MSE alone can favor overly complex models

Key point: To choose features fairly, evaluate performance using validation data, cross-validation, or a penalty for complexity.

Why training MSE can be misleading

Training MSE behaves like a kid with a credit card in a candy store:

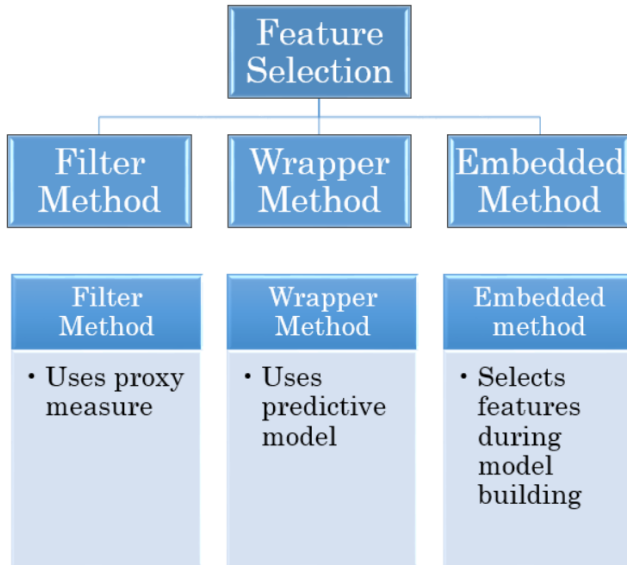
- It keeps rewarding models for adding more features
- Even weak or useless features can reduce training error (the store is sold out)
- There is no built-in penalty for model complexity

Problem: minimizing training MSE alone can lead to overfitting.

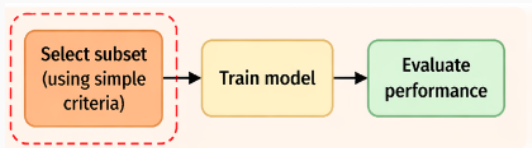


How do we educate this child?

Feature selection methods



Filter methods



- Select features **before** model training
- Use simple, model-independent criteria, such as:
 - Large absolute correlation with the outcome
 - Statistical tests, e.g. t-test, ANOVA, chi-square test
- Based on intrinsic feature properties, not model performance

Limitations

- Selected features may not work well together
- Ignore feature interactions
- Can be sensitive to multicollinearity

Example: House Price Prediction

House Price Prediction Using Melbourne Housing Data

- **Response variable y :**
 - **Price:** Selling price of the property
- **Potential predictors:**
 - **Rooms:** Total number of rooms
 - **Bedroom2:** Number of bedrooms
 - **Bathroom:** Number of bathrooms
 - **BuildingArea:** Building size (m^2)
 - **Landsize:** Land size (m^2)
 - **Distance:** Distance to Melbourne CBD (km)
 - **YearBuilt:** Year the property was built

Example: House Price Prediction, Melbourne housing data

- Select features with $|\text{correlation with Price}| > 0.4$
- Some relevant features with low marginal correlation are removed: Distance, YearBuilt, Landsize.
- Redundant features may be kept: Rooms and Bedroom2 which are highly correlated with each other.



Parameter estimates

- What is the effect of adding a bedroom while keeping total rooms fixed?
- This is a classic multicollinearity artifact, not a real negative effect.

Call:

```
lm(formula = Price ~ . - Distance - YearBuilt - LotSize, data = house)
```

Residuals:

Min	1Q	Median	3Q	Max
-3044257	-319024	-93140	235460	5309105

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	63124.2	42131.6	1.498	0.1342
Rooms	214054.6	46237.8	4.629	3.88e-06 ***
Bedroom2	-92123.1	46165.1	-1.996	0.0461 *
Bathroom	203324.1	23086.4	8.807	< 2e-16 ***
BuildingArea	2157.0	181.3	11.901	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

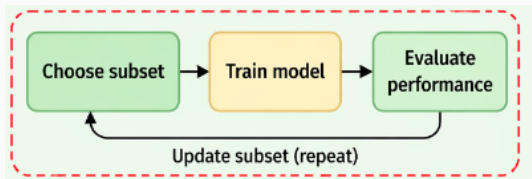
Residual standard error: 576500 on 2216 degrees of freedom

Multiple R-squared: 0.3153, Adjusted R-squared: 0.3141

F-statistic: 255.1 on 4 and 2216 DF, p-value: < 2.2e-16

Wrapper methods

Select while training



- Select features that perform well in the model, e.g.:
 - Feature selection is guided by model performance
 - The model is trained many times with different feature subsets
 - Examples: Forward selection, Backward elimination, Stepwise regression, Best subset selection
- Advantages: Accounts for feature interactions and avoids selecting redundant features
- Disadvantages: Computationally expensive, can overfit if not validated properly

Best subset selection

- Wrapper method evaluating **all feature subsets**
 - Fits a model for **each subset**
 - Final model selected using a **model-selection criterion** (e.g. validation or CV error, AIC, BIC)
-
- Number of models grows exponentially: 2^p
 - $p = 10 \rightarrow 1,024$ models
 - $p = 20 \rightarrow 1,048,576$ models

Forward/backward and stepwise selection

1. Start with an initial model
 - **Forward:** no predictors
 - **Backward:** full model
 - **Stepwise:** an intermediate model
2. Choose a fit criterion that penalizes complexity
 - AIC / BIC
 - Validation or cross-validation error
3. Evaluate one-feature changes
 - Try adding or removing one feature at a time
4. Update the model
 - Add or remove the feature that gives the largest improvement
5. Repeat until the criterion no longer improves

Note

- Greedy search instead of testing all subsets
- Much cheaper than best subset selection
- With 20 features, best subset selection checks $2^{20} = 1,048,576$ possible subsets, while stepwise selection usually evaluates far fewer models (max 211).

Stepwise selection in R: `step()`

```
step(object, scope, direction = c("both", "backward", "forward"), k = 2)  
)
```

`step()` performs **greedy feature selection** by optimizing a penalized fit criterion.

It uses **AIC/BIC-type criteria**, not cross-validation.

- `object`: fitted model, `lm()`, used as the starting point
- `scope`: range of models to search
 - `lower`: smallest model allowed
 - `upper`: largest model allowed
- `direction`: direction of the search
 - `"forward"`: add predictors
 - `"backward"`: remove predictors
 - `"both"`: add and remove predictors
- `k`: penalty for model complexity
 - `k = 2` gives AIC
 - `k = log(n)` gives BIC

Result backward stepwise house prices

```
step(lm(Price ~ ., house), trace = 0)
```

- Response: Price
- Predictors: all other variables in house.
- This is your starting model for stepwise selection.
- We didn't specify direction, `step()` defaults to backward selection when starting from a full model.
- `trace = 0` controls printing during the stepwise search. Without it, you would see:
 - which variables were dropped
 - how AIC changed at each step

Result backward stepwise house prices

This is the final model selected by backward stepwise selection using AIC.

Call:

```
lm(formula = Price ~ Rooms + Distance + Bathroom + BuildingArea +  
    YearBuilt + Landsize, data = house)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-3124641	-255996	-61919	187573	4467329

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.172e+07	6.209e+05	18.88	<2e-16 ***
Rooms	1.577e+05	1.552e+04	10.16	<2e-16 ***
Distance	-2.704e+04	1.647e+03	-16.41	<2e-16 ***
Bathroom	2.551e+05	1.981e+04	12.88	<2e-16 ***
BuildingArea	2.152e+03	1.519e+02	14.17	<2e-16 ***
YearBuilt	-5.879e+03	3.178e+02	-18.50	<2e-16 ***
Landsize	2.009e+01	7.973e+00	2.52	0.0118 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

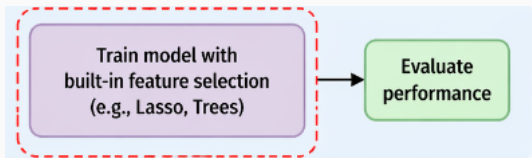
Residual standard error: 483100 on 2214 degrees of freedom

Multiple R-squared: 0.5196, Adjusted R-squared: 0.5183

F-statistic: 399.1 on 6 and 2214 DF, p-value: < 2.2e-16

Embedded methods

Regularization/shrinkage



Buy as many coefficients as you like, but with a limited budget

- Feature selection happens during model training
- coefficients are shrunk to fit the budget
 - Coefficients are shrunk toward zero
 - Some coefficients may become exactly zero
- Regularization discourages large coefficients instead of explicitly selecting subsets.

Regularization / shrinkage

Regularization fits the model using a **penalized loss function**:

$$\text{Loss} = \text{MSE} + \text{Penalty}$$

Equivalently:

$$\text{Loss} = \text{Training error} + \text{Penalty on coefficient size}$$

- Penalization limits model complexity
- Coefficients are shrunk toward zero
- More stable than wrapper methods
- Computationally cheaper than best subset selection
- Less prone to overfitting

Two well-known shrinkage methods: Lasso vs Ridge

Two regularization methods with **different penalty** functions:

$$\text{lasso penalty} = \lambda \sum_j |\beta_j|$$

- Shrinks coefficients

- Some coefficients become exactly zero
- Performs feature selection

$$\text{ridge penalty} = \lambda \sum_j \beta_j^2$$

- Shrinks coefficients toward zero
- No coefficients are set exactly to zero
- Keeps all features

λ is the tuning parameter that controls how strongly large coefficients are penalized.

Choosing the penalty strength λ

- Select the **optimal** λ using **cross-validation**
- Cross-validation tells us how much complexity the data can afford.
- Larger $\lambda \rightarrow$ **stronger shrinkage**
- Stronger shrinkage $\rightarrow$ **simpler model** (and **sparser**)

Regularization with package `glmnet`

Example ridge regression with functions `glmnet()` and `cv.glmnet()`:

- `glmnet()`: Fits regularized regression models (ridge, lasso, elastic net) for a grid of λ values
- `cv.glmnet()`: Uses cross-validation to select the optimal λ

R code example

```
# Fit ridge regression on the training data
train_ridge <- glmnet(
  x = as.matrix(Train[, -nr]),
  y = Train[, nr],
  alpha = 0          # alpha = 0 --> ridge (alpha = 1 --> lasso)
)

# Plot coefficient shrinkage paths
plot(train_ridge, label = TRUE)

# Cross-validation to choose lambda
cv_ridge <- cv.glmnet(
  x = as.matrix(Train[, -nr]),
  y = Train[, nr], alpha = 0,
  type.measure = "mse"
)

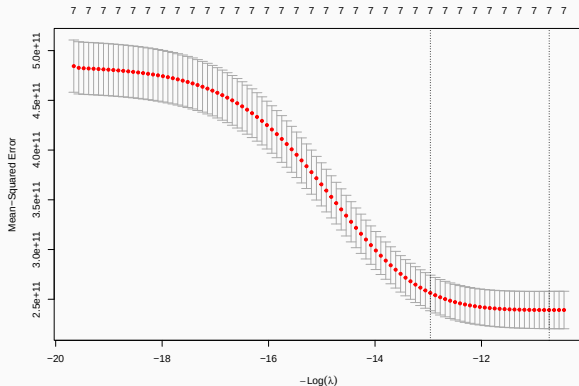
# Plot cross-validation error vs lambda
plot(cv_ridge)

# Predict on test data using optimal lambda
test_ridge <- predict(
  cv_ridge, newx = as.matrix(Test[, -nr]),
  s = "lambda.min"
)
```

Cross-validated MSE vs $\log(\lambda)$ for ridge regression

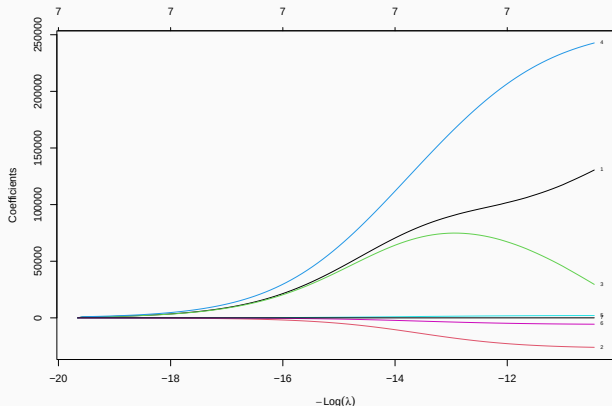
- optimal $\log(\lambda)$'s approximately at 10 and 12.5
- $\lambda_{\min}$: gives the lowest cross-validated MSE.
- λ_{1se} : gives stronger regularization with nearly the same MSE.

```
ridge_house <- cv.glmnet(x = as.matrix(house[, -1]),  
  y = unlist(house[, 1]), alpha = 0)  
plot(ridge_house)
```



Shrinkage plot for ridge: Ridge coefficient shrinkage paths

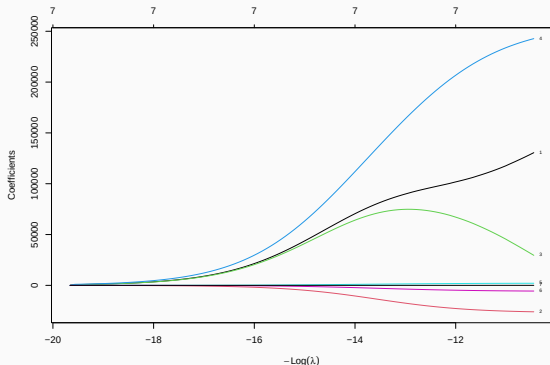
```
plot(glmnet(x = as.matrix(house[, -1]),  
          y = unlist(house[, 1]),  
          alpha = 0),  
     "lambda", label = T)
```



Interpretation of the coefficient plot for Ridge

- Right side (small λ) : Weak penalty, and coefficients can be large
- Left side (large λ): Strong penalty, coefficients are heavily shrunk
- Moving left to right means decreasing regularization. Ridge regression gradually shrinks all coefficients toward zero as the penalty increases.

```
plot(glmnet(x = as.matrix(house[, -1]), y = unlist(house[, 1]),  
          alpha = 0), "lambda", label = T)
```



Ridge coefficients

```
# ridge_house$lambda.min      # lambda with minimum CV error
# ridge_house$lambda.1se      # largest lambda within 1 SE of the minimum
# coef(ridge_house, s = "lambda.min")
# coef(ridge_house, s = "lambda.1se")
```

```
ridge_house$lambda.min
```

```
[1] 45631.47
```

```
coef(ridge_house, s = "lambda.min")
```

```
8 x 1 sparse Matrix of class "dgCMatrix"
```

	lambda.min
(Intercept)	1.104148e+07
Rooms	1.238428e+05
Distance	-2.572173e+04
Bedroom2	3.767244e+04
Bathroom	2.386997e+05
BuildingArea	2.044121e+03
YearBuilt	-5.522917e+03
Landsize	1.840856e+01

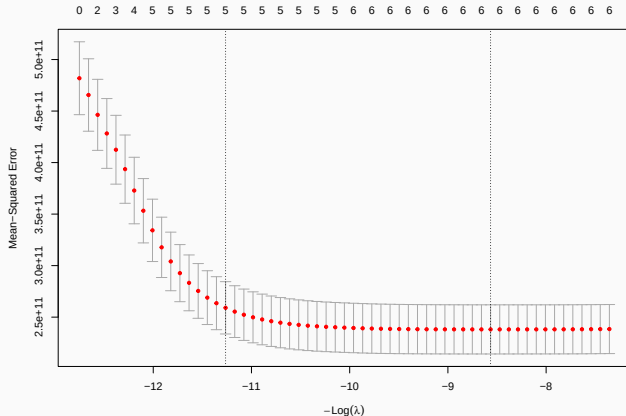
Ridge prediction

```
pred <- predict(ridge_house, newx = as.matrix(Test[, -1]), s = "lambda.min")
```


Cross-validation plot lasso

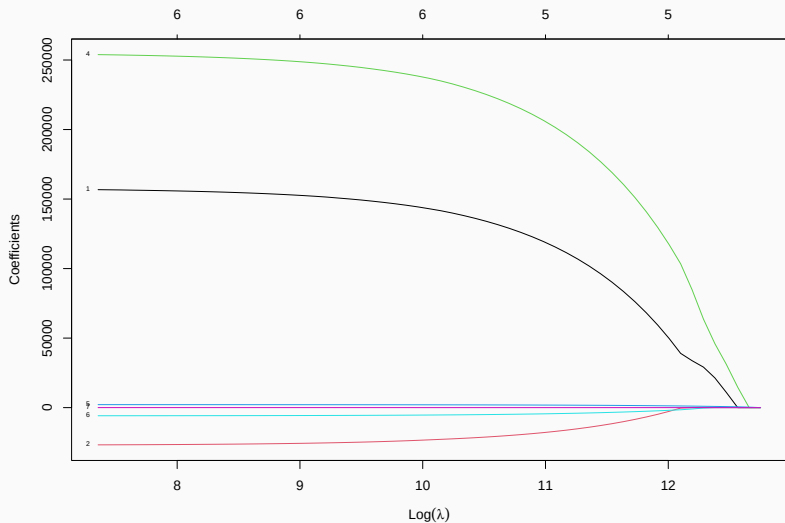
- optimal $\log(\lambda)$'s approximately at 7 and 11.5

```
plot(cv.glmnet(x = as.matrix(house[, -1]),  
              y = unlist(house[, 1]),  
              alpha = 1))
```



lasso shrinkage (coefficient path) plot

- number of features decreases as lambda increases



Ridge vs Lasso parameters

Which table corresponds to Ridge and which to Lasso?

7 x 2 sparse Matrix of class "dgCMatrix"

	s99	s71
Rooms	130538.0	89123.0
Distance	-26018.6	-17162.7
Bedroom2	29558.5	74727.9
Bathroom	242738.9	161258.0
BuildingArea	2071.6	1419.7
YearBuilt	-5607.5	-3574.5
Landsize	18.8	10.1

7 x 2 sparse Matrix of class "dgCMatrix"

	s45	s17
Rooms	154428.7	111411.6
Distance	-26166.4	-16202.9
Bedroom2	.	.
Bathroom	250997.9	196288.0
BuildingArea	2124.1	1773.8
YearBuilt	-5755.4	-4178.6
Landsize	15.8	.

How Ridge and Lasso Treat Variables Differently

- Ridge coefficients table
 - All coefficients are usually non-zero
 - Coefficients are shrunk toward zero, but not fully removed
 - Ridge keeps all variables in the model
- Lasso coefficients table
 - Some coefficients are exactly 0
 - Only a subset of variables remains
 - Performs automatic feature selection

Dimensionality reduction and PCA regression

Principal Components Analysis (PCA)

- Transforms (condense) the original features into a small number of new variables
- These new variables are called principal components (PCs)
- Each PC is a linear combination of the original features
- PCs are orthogonal (uncorrelated)
- Ordered by how much variance they explain

Example: Principal Components Analysis (PCA)

Items in a questionnaire:

x1: I like parties

x2: I like to be the center of attention

x3: I have many friends

x4: I worry a lot

x5: I am nervous

x6: I am irritable

Example: Principal Components Analysis (PCA)

x_1 : I like parties
 x_2 : I like to be the center of attention
 x_3 : I have many friends

→ **PC1 (Extraversion)**

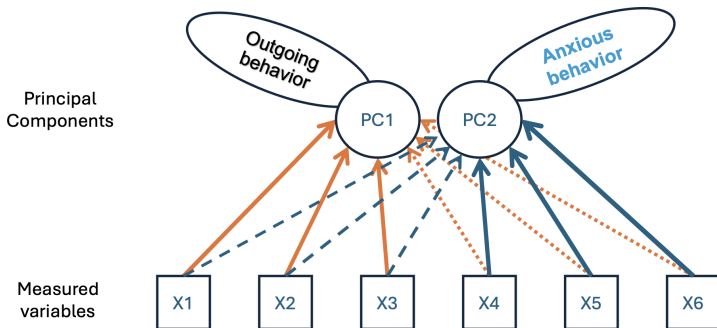
x_4 : I worry a lot
 x_5 : I am nervous
 x_6 : I am irritable

→ **PC2 (Neuroticism)**

The six items could probably be reduced to two variables

Example: Principal Components Analysis (PCA)

Data reduction



The six items could probably be reduced to two variables

How PCA is used for prediction

- We need to find PCs based on typical rules:
 - Elbow rule (Cattell, 1966)
 - Kaiser rule (eigenvalue > 1)
 - Cumulative variance threshold (e.g. 70 – 90)
- Keep only the first few PCs including high loading (more on this later).
- These explain most of the variability in the data
- Use the selected PCs as predictors in a model

PCA vs Regularization (Lasso and Ridge)

- PCA reduces dimensionality by changing the feature space, not by selecting features.
- Ridge and Lasso decide how much to use each feature.
- PCA decides to throw away the features entirely and build new ones.

What does a loading measure in PCA?

- A loading indicates how strongly an original variable is associated with a principal component.
- In standardized PCA, loadings can be interpreted approximately as the correlation between the variable and the principal component.
- Larger absolute loadings mean the variable contributes more strongly to that component.

- Package: `psych`
- Function: `principal()`

Example: PC for House price features

- Goal: Replace the original predictors with 3 principal components (PCs).

```
pc <- psych::principal(house[, -1], nfactors = 3)
pc$loadings
```

Loadings:

	RC1	RC2	RC3
Rooms	0.927		0.108
Distance	0.168	0.734	0.256
Bedroom2	0.928		0.105
Bathroom	0.793	0.139	
BuildingArea	0.785		
YearBuilt		0.871	-0.130
Landsize			0.962

	RC1	RC2	RC3
SS loadings	2.995	1.331	1.033
Proportion Var	0.428	0.190	0.148
Cumulative Var	0.428	0.618	0.766

- `house[, -1]`: uses all predictor variables (drops the first column, e.g. Price)
- `nfactors = 3`: compute the first 3 principal components
- `pc$loadings`: shows the weights (loadings) used to build each PC from the original variables

Example: PC for House price features

- PC1 (Space): High loadings on Rooms, Bedrooms2, Bathroom, BuildingArea
- PC2 (Time): High loadings on YearBuilt, Distance
- PC3 (Land): High loading on Landsize
- A large loading means the variable strongly contributes to that PC.
- A positive or negative sign shows the direction of the relationship.
- Variables with large loadings on the same PC tend to represent a shared pattern.

Loadings:

	RC1	RC2	RC3
Rooms	0.927		0.108
Distance	0.168	0.734	0.256
Bedroom2	0.928		0.105
Bathroom	0.793	0.139	
BuildingArea	0.785		
YearBuilt		0.871	-0.130
Landsize			0.962

	RC1	RC2	RC3
SS loadings	2.995	1.331	1.033
Proportion Var	0.428	0.190	0.148
Cumulative Var	0.428	0.618	0.766

PCs:

$PC_1 = 0.927, \text{Rooms} + 0.928, \text{Bedroom2} + 0.793, \text{Bathroom} + 0.785, \text{BuildingArea} + 0.168, \text{Distance}$

$PC_2 = 0.871, \text{YearBuilt} + 0.734, \text{Distance} + 0.139, \text{Bathroom}$

$PC_3 = 0.962, \text{Landsize} + 0.256, \text{Distance} + 0.108, \text{Rooms} - 0.130, \text{YearBuilt}$

Linear model with the three components: R code

```
summary(lm(house$Price ~ ., data = as.data.frame(pc$scores)))
```

Call:

```
lm(formula = house$Price ~ ., data = as.data.frame(pc$scores))
```

Residuals:

Min	1Q	Median	3Q	Max
-2657313	-283958	-61201	193252	4875752

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1100262	10576	104.038	<2e-16 ***
RC1	401035	10578	37.912	<2e-16 ***
RC2	-274725	10578	-25.972	<2e-16 ***
RC3	9116	10578	0.862	0.389

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 498400 on 2217 degrees of freedom

Multiple R-squared: 0.4879, Adjusted R-squared: 0.4873

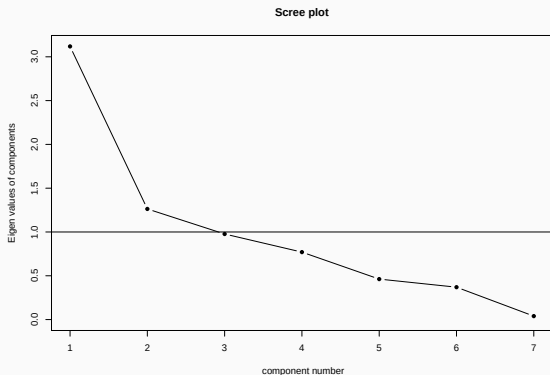
F-statistic: 704.2 on 3 and 2217 DF, p-value: < 2.2e-16

Remark: Methods for choosing number of PCs:

- Scree plot (elbow rule)
- Cumulative variance explained
- Kaiser criterion (eigenvalue > 1)
- Cross-validation (for prediction)
- Domain knowledge + interpretability

Scree Plot: Choosing the Number of PCs

```
psych::scree(house[, -1], factors = FALSE) # without FA
```



- The eigenvalue shows how much variance each PC explains.
- The **elbow rule**: keep PCs before the curve starts to flatten.
- After the elbow, additional PCs add relatively little new information.

Takeaways 1: Feature Selection

- **Feature selection helps control model complexity**
 - Too few features → underfitting
 - Too many features → overfitting
 - Good feature selection improves prediction, interpretability, and stability
- **Different methods select features in different ways**
 - **Filter methods:** select features before modelling
 - **Wrapper methods:** test different feature subsets, e.g. stepwise selection
 - **Embedded methods:** select or shrink features during model training, e.g. lasso and ridge

Takeaways 2: Regularization & PCA

- **Regularization controls coefficient size**
 - **Ridge:** shrinks coefficients but keeps all variables
 - **Lasso:** can shrink some coefficients exactly to zero
- **PCA is dimensionality reduction, not feature selection**
 - It replaces original variables with new variables called **principal components**
 - PCs are weighted combinations of the original features
 - Keep the first few PCs using rules such as the scree plot, Kaiser rule, or variance explained
 - PCs can also be used for predictions.

Feature selection keeps useful original predictors; PCA creates fewer new predictors that summarize the original data.

Predict the graduation rate of large number of US colleges from 17 features

Perform feature selection methods

- filter, wrapper, and regularization